

CICLOS en PROGRAMACION

Sosa Martinez Danna Michelle

Programacion

Turno: Matutino



Colegio De Estudios Cientificos Y
Tecnologicos Del Estado De Mexico

Grupo:

205

INTRODUCCION

¿Que es un ciclo en programacion?

En programacion, un ciclo (tambien llamado bucle) es una estructura de control que permite ejecutar un bloque de codigo repetidamente hasta que se cumpla una condicion especifica. Son fundamentales para

automatizar tareas repetitivas y optimizar el código, ya que evitan la necesidad de escribir el mismo código varias veces.

¿Por qué son importantes los ciclos en programación?

Los ciclos (o bucles) en programación son cruciales porque permiten ejecutar un bloque de código repetidamente, lo que ahorra tiempo y esfuerzo al evitar la necesidad de reescribir el mismo código varias veces. Los ciclos también son esenciales para realizar tareas repetitivas de manera eficiente y para crear programas más legibles y mantenibles, los ciclos son una herramienta esencial para cualquier programador, ya que permiten crear código más eficiente, legible y fácil de mantener, especialmente al manejar tareas repetitivas y controlar el flujo de ejecución.

CICLO FOR

Fundamentos Básicos

El ciclo "for" en programación es una estructura que permite ejecutar un bloque de código repetidamente un número definido de veces. Se caracteriza por tres partes principales: inicialización de una variable, condición de continuación y actualización de la variable. Esta estructura es útil para tareas que requieren repetición de acciones con un contador que se incrementa o decrementa.

La instrucción for permite repetir una instrucción o una instrucción compuesta un número especificado de veces. El cuerpo de una instrucción for se ejecuta cero o más veces hasta que una condición opcional sea false.

Sintaxis

La sintaxis básica de un ciclo for en programación generalmente sigue el formato: `for (inicialización; condición; incremento) { cuerpo del bucle; }`. Este ciclo repite un bloque de código un cierto número de veces, controlado por una variable de iteración.

1. Inicialización:

Esta parte se ejecuta solo al inicio del bucle, antes de que comience la iteración. Aquí se define una variable de iteración (a menudo llamada *i*, *j*, etc.) y se le asigna un valor inicial.

2. Condición:

Esta parte se evalúa antes de cada iteración del bucle. Si la condición es verdadera, se ejecuta el cuerpo del bucle. Si es falsa, el bucle termina.

3. Incremento/Decremento:

Después de cada iteración del bucle, se ejecuta esta parte. Normalmente, se utiliza para actualizar la variable de iteración (aumentar o disminuir su valor).

4. Cuerpo del bucle:

Este es el bloque de código que se ejecutará repetidamente mientras la condición sea verdadera.

Ventajas

- **Simplificación de código repetitivo:**

El ciclo "for" elimina la necesidad de escribir el mismo código varias veces para ejecutar una tarea repetidamente. Permite repetir un bloque de código un número determinado de veces, ahorrando tiempo y haciendo el código más conciso.

- **Iteración a través de colecciones:**

El ciclo "for" es ideal para recorrer elementos en colecciones como arrays o listas. Permite acceder a cada elemento de la coleccion en orden, lo que facilita tareas como imprimir, modificar o procesar cada elemento.

- **Mayor legibilidad:**

Al utilizar el ciclo "for" en lugar de bucles "while" o estructuras de control mas complejas, el codigo se vuelve mas legible y facil de entender. La estructura clara y sintaxis concisa del "for" facilita la comprension de la logica de iteracion.

- **Control de iteraciones:**

El ciclo "for" proporciona un mecanismo para controlar el numero de iteraciones de forma precisa. La declaracion inicial, la condicion de continuacion y el incremento/decremento permiten definir el comportamiento del bucle de manera controlada.

- **Mejora la eficiencia:**

En situaciones donde se conoce el numero de iteraciones, el ciclo "for" puede ser mas eficiente que otros tipos de bucles, ya que la condicion de continuacion y el incremento/decremento estan claramente definidos.

Cuando Se Recomienda Usar La Funcion For

El ciclo for en programacion se recomienda utilizar cuando se conoce de antemano el numero de veces que se debe ejecutar un bloque de codigo. Esto ocurre, por ejemplo, al iterar sobre una lista o matriz de elementos, o cuando se necesita ejecutar un codigo un numero especifico de veces.

Ejemplo Practico

si quieres imprimir una lista de numeros del 1 al 10, puedes usar un ciclo "for" para evitar tener que escribir la misma instruccion 10 veces.

Python

Ejemplo en Python

Imprimir los numeros del 1 al 10

```
for i in range(1, 11):  
    print(i)
```

Explicacion:

1. 1. for i in range(1, 11):

Esta linea inicia el ciclo "for".

- range(1, 11) crea una secuencia de numeros del 1 al 10 (el 11 no se incluye).
- i es una variable que tomara cada valor de la secuencia (1, 2, 3, ..., 10).

2. 2. print(i):

Esta linea imprime el valor actual de la variable i en cada iteracion del ciclo.

En resumen: El ciclo "for" repite el codigo dentro de el 10 veces, una por cada numero de la secuencia. En cada iteracion, i tomara un valor diferente (1 a 10) y ese valor se imprimira.

CICLO WHILE

Fundamentos Basicos

En programacion, un ciclo WHILE (bucle while) es una estructura de control que permite ejecutar un bloque de codigo repetidamente mientras una condicion especifica sea verdadera. La condicion se evalua antes de cada iteracion, y si es falsa, el ciclo se detiene.

Si la condicion nunca se vuelve falsa, el ciclo se ejecutara indefinidamente, lo que se conoce como un ciclo infinito. Para evitar esto, es importante que dentro del ciclo se realice alguna accion que eventualmente cambie el valor de la condicion para que se haga falsa.

Sintaxis

La sintaxis general de un ciclo while en programacion es `while (condicion) { // codigo a ejecutar }`. Este ciclo se ejecuta repetidamente mientras la condicion sea verdadera. Si la condicion es inicialmente falsa, el codigo dentro del ciclo nunca se ejecuta.

1. Si la condicion es inicialmente falsa, el codigo dentro del ciclo nunca se ejecuta. `while`: Palabras clave que indica el inicio del ciclo.
2. `(condicion)`: Una expresion booleana que se evalua. Si la condicion es verdadera, el codigo dentro del ciclo se ejecuta.
3. `{ // codigo a ejecutar }`: Un bloque de codigo que se repite mientras la condicion sea verdadera. Este bloque puede contener una o mas sentencias.

Ventajas

Los bucles desempeñan un papel crucial en el control del flujo de un programa, especialmente cuando no se conoce de antemano el número de iteraciones ni la duración de las acciones repetidas. Proporcionan flexibilidad y permiten la ejecución de tareas específicas hasta que se cumpla la condición deseada, lo que otorga a los programadores un mayor control sobre su código.

Los bucles While se pueden utilizar en diversos escenarios, como por ejemplo:

- Leer y procesar datos de fuentes externas hasta que se cumpla una condición específica.
- Iterar sobre elementos de una lista o matriz para realizar cálculos o manipulaciones.

- Construir programas interactivos que respondan a la entrada del usuario hasta que se cumpla una condicion especifica.
- Implementar la logica del juego que actualiza continuamente el estado del juego hasta que una determinada condicion activa una salida.

Cuando Se recomienda Usar La Funcion While

El ciclo while en programacion se usa cuando quieres ejecutar un bloque de codigo repetidamente mientras una condicion especifica sea verdadera. Esto es util cuando no sabes con exactitud cuantas veces el codigo debe ejecutarse, y la repeticion depende de la condicion que se este verificando.

En detalle:

- **Condicion de repeticion:**

El ciclo while verifica una condicion antes de cada iteracion del bucle. Si la condicion es verdadera, el codigo dentro del ciclo se ejecuta. Si la condicion es falsa, el ciclo termina.

- **Numero indefinido de repeticiones:**

A diferencia de los ciclos for que suelen tener un numero predefinido de iteraciones, el while es ideal cuando la condicion que determina la repeticion es dinamica y puede cambiar a lo largo de la ejecucion.

- **Ejemplos de uso:**

- **Leer datos de entrada:** Puedes usar un while para leer datos de entrada del usuario hasta que se ingrese un valor especifico o se cumpla una condicion.

- **Procesar elementos de una lista:** Si necesitas procesar elementos de una lista hasta que se encuentre un elemento específico o se cumpla una condición relacionada con el contenido de la lista.

- **Simulaciones:** En simulaciones donde el número de pasos o iteraciones no se conoce de antemano, el ciclo `while` puede ser útil.

Ejemplo Practico

Imagina un programa que pide al usuario un número y lo valida hasta que se ingrese un número válido (entre 1 y 10).

Python

Ejemplo de validación de entrada

`while`

```
numero = int(input("Ingresa un número entre 1 y 10: "))
while True:
    if 1 <= numero <= 10:
        print("Número válido:", numero)
        break # Salir del bucle si la entrada es correcta
    else:
        print("Número no válido. Intentalo de nuevo.")
except ValueError:
    print("Entrada inválida. Debes ingresar un número.")
```

En este ejemplo, el bucle `while True` se ejecutará indefinidamente hasta que el usuario ingrese un número válido. El `break` se usa para salir del bucle cuando se cumple la condición de validación.

En resumen, el ciclo `while` es una herramienta útil para repetir bloques de código mientras se cumple una condición, permitiendo realizar tareas repetitivas de forma eficiente en la programación.

CICLO DO-WHILE

Fundamentos Basicos

El ciclo do-while en programacion es un bucle que garantiza que un bloque de codigo se ejecute al menos una vez antes de evaluar una condicion. A diferencia del bucle while, que evalua la condicion antes de ejecutar el bloque, el do-while ejecuta el bloque y luego evalua la condicion.

Sintaxis

La sintaxis de un ciclo do-while en programacion generalmente sigue este formato: `do { // Bloque de codigo que se ejecutara al menos una vez } while (condicion);` La principal característica es que el bloque de codigo dentro de do se ejecuta al menos una vez, antes de que se evalúe la condicion en while.

Explicacion detallada:

1. `do { ... }:`

Esta parte marca el inicio del bloque de codigo que se ejecutara repetidamente. El codigo dentro de las llaves `{}` se ejecutara al menos una vez, sin importar el valor de la condicion inicial.

2. `while (condicion):`

Esta parte evalua una condicion booleana. Si la condicion es verdadera, el bloque de codigo dentro de do se vuelve a ejecutar. Si la condicion es falsa, el ciclo termina.

3. `::El punto y coma`

`;` Al final de la instruccion while es importante para indicar el final de la estructura do-while.

Ventajas

La principal ventaja del ciclo do-while en programación es que garantiza que el bloque de código dentro del ciclo se ejecute al menos una vez, incluso si la condición de salida es falsa desde el principio.

- **Ejecución garantizada:**

El código dentro del do-while se ejecuta antes de que se evalúe la condición, por lo que se ejecuta al menos una vez, independientemente de si la condición es verdadera o falsa.

- **Ideal para entrada de datos:**

Es útil cuando se necesita solicitar datos al usuario y reintentar la solicitud hasta que se ingrese un valor válido, asegurando que la solicitud al usuario se muestre al menos una vez.

- **Sencillez y claridad:**

Su estructura simple y directa lo hace fácil de entender y mantener, especialmente para situaciones donde la ejecución inicial es obligatoria.

- **Alternativa a otros ciclos:**

En algunos casos, el do-while puede ser una alternativa más conveniente que el while o el for cuando se necesita asegurar la ejecución inicial de un bloque de código.

Cuando Se recomienda Usar La Funcion Do-While

El ciclo do-while en programación es ideal cuando necesitas asegurar que un bloque de código se ejecute al menos una vez y luego se repita mientras se cumple una condición. A diferencia de while, donde la condición

se evalua antes de ejecutar el código, en do-while, el código se ejecuta primero y luego se verifica la condición.

- **Validación de entrada del usuario:**

Cuando necesitas que el usuario introduzca datos válidos antes de continuar con el programa, el do-while es perfecto para solicitar la entrada repetidamente hasta que sea correcta.

- **Menús de interacción con el usuario:**

En menús donde el usuario debe seleccionar una opción antes de que se pueda continuar, el do-while garantiza que el usuario vea el menú y pueda elegir una opción al menos una vez.

- **Cuando necesitas asegurar la ejecución inicial:**

Si necesitas que un bloque de código se ejecute sin importar la condición inicial, el do-while es la mejor opción.

- **Cuando no conoces el número de iteraciones pero necesitas la ejecución inicial:**

Si no sabes cuántas veces se repetirá el ciclo, pero sabes que se ejecutará al menos una vez, el do-while es adecuado.

Ejemplo Práctico

Un ejemplo práctico del ciclo do-while en programación podría ser la creación de un menú interactivo en un juego o programa, donde se muestra al usuario una lista de opciones y se le pide que elija una. Este ciclo asegura que el menú se muestre al menos una vez antes de que el usuario pueda interactuar con él y elegir una opción.

Ejemplo en C++:

C++

```
#include<iostream>
```

```
#include<string>
```

```
int main(){
```

```
    int opcion;
```

```
do{
```

```
    //Mostrar el menu
```

```
    std::cout<<"\nMenu principal:"<<std::endl;
```

```
    std::cout<<"1.Jugar"<<std::endl;
```

```
    std::cout<<"2.Configuracion"<<std::endl;
```

```
    std::cout<<"3.Salir"<<std::endl;
```

```
    std::cout<<"Seleccione una opcion:";
```

```
    //Leer la opcion del usuario
```

```
    std::cin>>opcion;
```

```
    //Procesar la opcion
```

```
    switch(opcion){
```

```
        case 1:
```

```
            std::cout<<"Iniciando el juego..."<<std::endl;
```

```
            //...
```

```
            break;
```

```
        case 2:
```

```
            std::cout<<"Abriendo configuracion..."<<std::endl;
```

```
            //...
```

```
            break;
```

```
        case 3:
```

```
            std::cout<<"Saliendo del programa..."<<std::endl;
```

```
            break;
```

```
        default:
```

```
            std::cout<<"Opcion invalida. Intente de nuevo."<<std::endl;
```

```
}  
} while (opcion != 3); // Salir cuando se elija la opcion 3
```

```
return 0;  
}
```

Explicacion:

1. do {...} while (condicion):

Este ciclo ejecuta el bloque de código dentro de las llaves do al menos una vez. Luego, evalúa la condición while (condicion); al final del ciclo. Si la condición es verdadera, el bucle se ejecuta nuevamente. Si es falsa, el bucle termina.

2. Mostrar el menú:

El código dentro del do muestra el menú al usuario con las diferentes opciones disponibles.

3. Leer la opción:

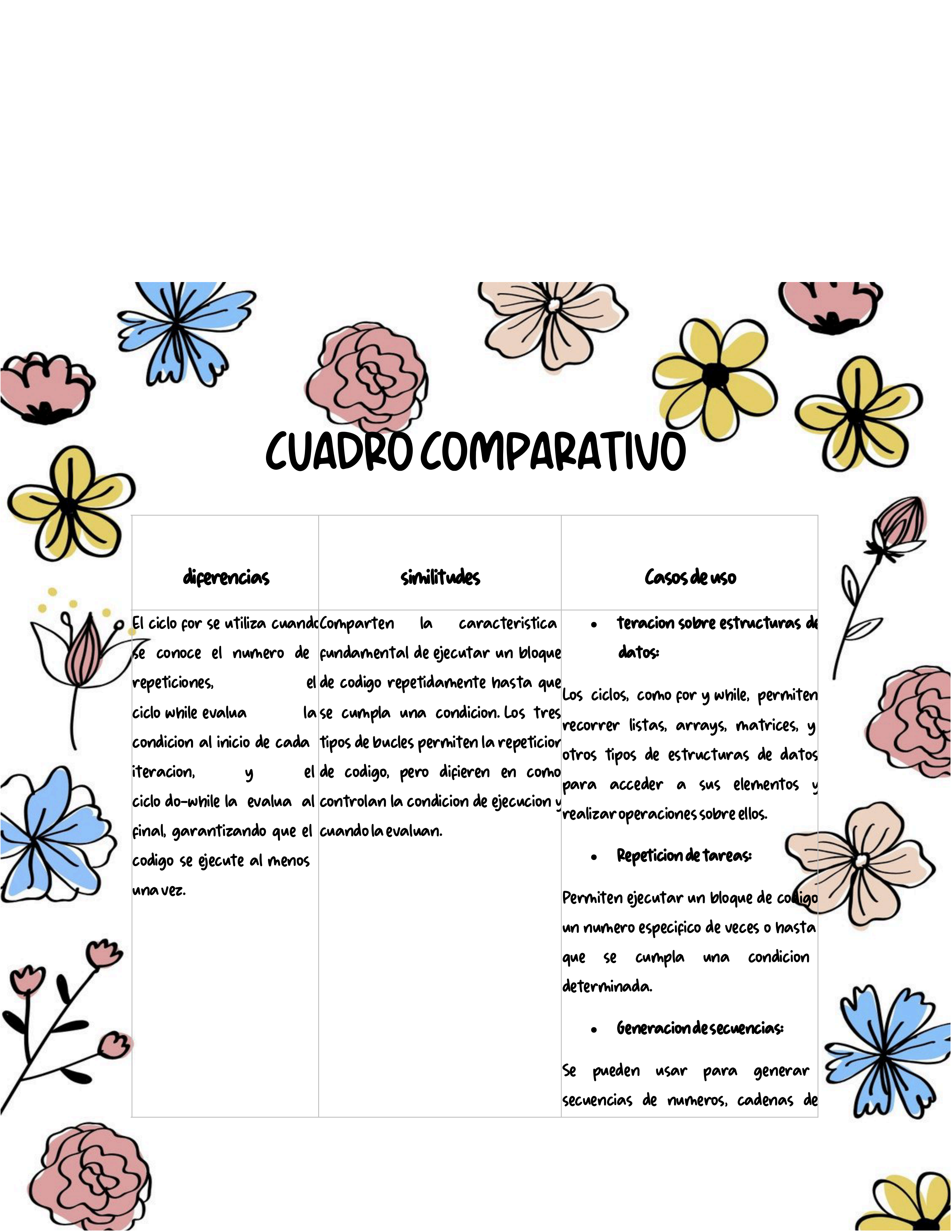
El usuario ingresa su opción (1, 2, 3, etc.).

4. Procesar la opción:

Un switch (o una serie de if/else if/else) procesa la opción elegida por el usuario, realizando las acciones correspondientes (ejecutar el juego, abrir la configuración, salir, etc.).

5. Condición de salida:

La condición while (opcion != 3); controla cuando el bucle debe terminar. En este caso, el bucle se ejecuta hasta que el usuario ingrese la opción 3 (Salir).



CUADRO COMPARATIVO

diferencias	similitudes	Casos de uso
El ciclo for se utiliza cuando se conoce el número de repeticiones, ciclo while evalúa la condición al inicio de cada iteración, y ciclo do-while la evalúa al final, garantizando que el código se ejecute al menos una vez.	Comparten la característica fundamental de ejecutar un bloque de código repetidamente hasta que se cumpla una condición. Los tres tipos de bucles permiten la repetición de código, pero difieren en cómo controlan la condición de ejecución y cuándo la evalúan.	• iteración sobre estructuras de datos: Los ciclos, como for y while, permiten recorrer listas, arrays, matrices, y otros tipos de estructuras de datos para acceder a sus elementos y realizar operaciones sobre ellos. • Repetición de tareas: Permiten ejecutar un bloque de código un número específico de veces o hasta que se cumpla una condición determinada. • Generación de secuencias: Se pueden usar para generar secuencias de números, cadenas de

texto, u otros patrones.

REFLEXION PERSONAL

¿Cual te parece mas util y por que?

El ciclo "for" , ya que suele ser mas versatil para distintos tipos de pseudocodigos, a diferencia de las funciones "while" y "do-while" ya que estos son mas especificos respecto a los programas en los que se van a emplear estos ciclos.

BIBLIOGRAFIA

<https://www.ni.com/es/support/documentation/supplemental/08/labview-for-loops-and-while-to-ops-explained.html#:~:text=Regresar%20al%20Inicio-,Casos%20de%20uso%20para%20cada%20estructura,cu%C3%A1ndo%20debe%20detenerse%20el%20c%C3%B3digo>

<https://es.scribd.com/document/468891985/Que-son-los-ciclos-de-programacion-y-como-funcionan-docx#:~:text=n%C3%BAmero%20de%20repeticiones-,Existen%20tres%20tipos%20principales:%20for%20se%20usa%20cuando%20se%20conoce,c%C3%B3digo%20m%C3%A1s%20eficiente%20y%20legible.>

https://support.google.com/websearch/answer/14901683?hl=es-419&visit_id=638814980099306666-3067276553&p=ai-overviews&rd=1

<https://www.shiksha.com/online-courses/articles/introduction-do-while-loop-in-c/>

<https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/loop-do-while-de-java/#:~:text=Syntax%20do%2Dwhile%20en%20Java&text=La%20estructura%20comienza%20con%20la,inicia%20la%20condici%C3%B3n%20de%20terminaci%C3%B3n.>

https://www2.eii.uva.es/fund_inf/cpp/temas/6_control_flujo_iterativo/do_while.html#:~:text=Es%20una%20variante%20especial%20de,ejecutar%C3%A1%20al%20menos%20una%20vez.

https://support.google.com/websearch/answer/14901683?hl=es-419&visit_id=638814980100923906-3031124630&p=ai-overviews&rd=1

https://support.google.com/websearch/answer/14901683?hl=es-419&visit_id=638814980100923906-3031124630&p=ai-overviews&rd=1

https://support.google.com/websearch/answer/14901683?hl=es-419&visit_id=638814980100923906-3031124630&p=ai-overviews&rd=1

